SYSTEM DOCUMENTATION & DEVELOPER GUIDE

Academic Projects Portfolio | Jan Michael M. Unarce

Purpose and Scope

This guide presents two academic software projects included in the portfolio: the Equipment Borrowing Management System and the AR Hand & Face Tracker – Boss Fight. It summarizes their purpose, technologies, main workflows, and development considerations. It is a portfolio-level guide, not a substitute for the complete source code, database schema, deployment configuration, or project-specific setup files.

Project 1 — Equipment Borrowing Management System

1.1 Overview

The Equipment Borrowing Management System is a web-based academic project intended to organize equipment borrowing requests and help administrators and staff monitor equipment transactions. It uses role-based access so users can interact with the system according to their assigned role.

1.2 Technology Stack

- Backend: Python with Django.
- Database: MySQL (database name used in the project plan: equipment_system).
- Authentication: Custom user model under the accounts application (AUTH_USER_MODEL = accounts.User).
- Application timezone: Asia/Manila.

1.3 Roles and Access

- Administrator: accesses the main dashboard at /dashboard/.
- Staff: accesses the staff dashboard at /dashboard/staff/.
- User: accesses the user dashboard at /dashboard/user/.

Role access is intended to be enforced through the application's authentication and role-access middleware. The exact permission matrix should be checked against the final source code before public release.

1.4 Borrowing Workflow and Business Rules

- A user submits a borrowing request, which begins with the pending status.
- A request may be approved or rejected by an authorized role.
- When approved, the planned due date is seven days after approval.
- The request lifecycle includes pending, approved, returned, overdue, and rejected statuses.
- The planned overdue penalty is ₱10 per day. Confirm the final calculation and when it is applied in the implemented code.
- The system includes unread notifications as a planned/implemented feature; verify the notification triggers and read-state behavior in the final build.

1.5 Main Components

- Authentication and roles: user sign-in and role-based access.
- Equipment records: equipment information used by the borrowing process.
- Borrow requests: request submission, review, status changes, and return tracking.

- Notifications: indicators for unread user notifications.
- Dashboards: role-specific views for administrative, staff, and user activities.

1.6 Configuration and Local Setup

The following is a general Django setup sequence. Use the project's actual requirements file, settings, and database credentials; do not copy example credentials into a public repository.

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
python manage.py makemigrations
python manage.py migrate
python manage.py runserver
```

Before running migrations, create the MySQL database and configure the connection in the project's settings or environment variables. Confirm the supported Django version from the actual requirements file; the project's development notes have referenced different Django versions, so this guide does not declare a verified version.

1.7 Testing Checklist

- Confirm that each role can sign in and is directed to the correct dashboard.
- Confirm that users cannot access pages or actions outside their assigned permissions.
- Submit a borrowing request and verify the pending status.
- Approve and reject requests; verify that status changes are saved correctly.
- Verify the seven-day due-date rule after approval.
- Test return processing and overdue calculations, including the ₱10/day rule.
- Verify unread notification creation, display, and read-state updates.
- Test invalid inputs, unauthorized access, and database error handling.

Testing status: Results have not been supplied for this document. Record actual test cases, expected results, observed results, and pass/fail outcomes after testing the current build.

Project 2 — AR Hand & Face Tracker – Boss Fight

2.1 Overview

AR Hand & Face Tracker – Boss Fight is a Python computer-vision project that combines hand tracking and face tracking with a simple interactive boss-fight experience. The camera feed is used to detect a player's hand and face, while on-screen game elements provide feedback such as the player indicator, boss health, shield, and score.

2.2 Technology Stack

- Language: Python.
- Computer vision: OpenCV for camera input and image display/processing.
- Hand tracking: MediaPipe Tasks HandLandmarker using the hand_landmarker.task model asset.
- Face tracking: face-tracking/mesh functionality integrated into the project.
- Runtime input: webcam/camera source, with the development configuration using camera index 0.

2.3 Main System Flow

- The application opens the camera and reads frames for processing.
- The hand-tracking component detects hand landmarks and uses gesture/position information as game input.
- The face-tracking component detects the player's face and draws visual tracking feedback.
- The game logic updates the boss encounter, including player actions and boss state.
- The display renders the camera view with augmented visual elements and game information.

2.4 Interaction and Controls

- The left index finger/hand interaction is part of the attack mechanic; the exact gesture threshold is configurable in the source.
- The R key is used to reset the game.
- The Q or Esc key is used to quit.
- The development version has included a pinch-distance threshold and multiple attack effects. Verify the final gesture mapping and any additional controls in the current code before documenting them as final behavior.

2.5 Important Files and Assets

- AR_Hand_Face_Tracker.py — combined application entry point used in the project.
- hand_landmarker.task — MediaPipe hand-landmark model asset required by the hand-tracking component.
- Python packages and model assets must be installed or placed where the source code expects them.

2.6 Running and Troubleshooting

- Use the Python environment and dependency versions compatible with the project's installed OpenCV and MediaPipe packages.
- Keep hand_landmarker.task in the expected path, or update the model path in the source code.
- Check camera permissions and close other applications that may be using the webcam.
- If the camera does not open, verify the camera index and try the correct device index.
- If tracking is slow, check camera resolution, frame-processing workload, and the number of enabled tracking features before changing the program structure.

2.7 Testing Checklist

- Confirm that the webcam opens and frames are displayed.
- Check hand detection under different lighting and hand positions.
- Check face detection and tracking feedback.
- Test the attack gesture and confirm the intended game response.

- Test boss health, shield, score updates, and reset behavior.
- Test quit keys and confirm the camera is released when the application closes.
- Observe frame rate and responsiveness on the target computer; record measured results rather than estimating them.

Testing status: No measured frame rate, accuracy, or formal test results have been provided. Add screenshots, test conditions, and actual observations from the final running version.

Documentation and Release Notes

Before publishing this guide, confirm that the described features match the final source code. Add the actual GitHub repository and demo links in the portfolio, include screenshots of each project, and remove any unused or outdated features from this document. Do not publish database passwords, API keys, private user data, or machine-specific paths.

Suggested Evidence to Attach

- Equipment Borrowing System: login/dashboard screenshots, borrowing request list, approval/return workflow, and notification view.
- AR Hand & Face Tracker: camera tracking screenshot, hand gesture/attack screenshot, and boss-fight result screen.
- For both projects: repository link, setup instructions, test cases, and a short description of your individual contribution.